

# LightGBM — Detaylı Konu Anlatımı

## Sınav Çalışma Notları

### Contents

|                                                             |   |
|-------------------------------------------------------------|---|
| 1. LightGBM Nedir? Motivasyon                               | 1 |
| 2. Leaf-wise (Best-First) Ağaç Büyütme — Temel Fark         | 1 |
| 3. Histogram Tabanlı Bölünme Arama                          | 2 |
| 4. GOSS — Gradient-based One-Side Sampling                  | 2 |
| 5. EFB — Exclusive Feature Bundling                         | 3 |
| 6. LightGBM’de Zaman Serisi Verisi — Kritik Bir Uyarı       | 3 |
| 7. Önemli Hiperparametreler — Sınav İçin Özet Tablo         | 3 |
| 8. LightGBM’in XGBoost ile Karşılaştırması                  | 4 |
| 9. LightGBM’in Genel Ensemble/Boosting Ailesi İçindeki Yeri | 4 |
| 10. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma        | 5 |

## 1. LightGBM Nedir? Motivasyon

**LightGBM (Light Gradient Boosting Machine)**, Microsoft tarafından geliştirilen, **histogram tabanlı** ve özellikle **çok büyük veri setlerinde hızlı eğitim** için optimize edilmiş bir Gradient Boosting çerçevesidir. XGBoost ile aynı temel matematiksel çerçeveyi (gradyan + Hessian tabanlı ikinci derece optimizasyon, regularizasyon) paylaşır, ancak **ağaç büyütme stratejisi ve örnekleme teknikleri** bakımından belirgin şekilde farklılaşır.

**Ne Zaman Tercih Edilir?** Çok sayıda satır ve/veya çok sayıda özellik içeren, eğitim süresinin kritik olduğu veri setlerinde (örn. yüzbinlerce-milyonlarca satır) LightGBM, XGBoost’a göre genelde belirgin şekilde daha hızlıdır ve daha az bellek kullanır.

---

## 2. Leaf-wise (Best-First) Ağaç Büyütme — Temel Fark

**Geleneksel Yaklaşım — Level-wise (Katman Katman) Büyütme:** Bir ağaç kurulurken, aynı derinlikteki (level) tüm düğümler **eş zamanlı** bölünür — önce derinlik 1’deki tüm düğümler, sonra derinlik 2’deki tüm düğümler bölünür. XGBoost’un varsayılan davranışı budur.

**LightGBM'in Yaklaşımı — Leaf-wise (Best-First) Büyütme:** Ağaç, derinlik sırasına göre değil, **en yüksek kayıp azalmasını (gain) sağlayacak yaprak** öncelikli olarak bölünür. Yani her adımda, mevcut tüm yapraklar arasından **en çok bilgi kazancı sağlayacak olan** seçilip bölünür.

**Sonuç:**

- Leaf-wise büyüme, aynı sayıda yaprak için genelde **daha düşük kayıp (daha doğru)** ağaçlar üretir.
- Ancak bu ağaçlar **dengesiz (asimetrik) ve daha derin** olabilir — bu da **küçük veri setlerinde overfitting riskini artırır**.
- Bu riski kontrol etmek için num\_leaves (yaprak sayısı üst sınırı) ve max\_depth parametreleri **dikkatli ayarlanmalıdır** — LightGBM'de num\_leaves, max\_depth'ten daha kritik bir hiperparametredir.

---

### 3. Histogram Tabanlı Bölünme Arama

LightGBM, sürekli değişkenleri eğitim öncesinde **ayrık kutulara (bin)** dönüştürür (genelde 255 kutu civarı):

1. Her sürekli özellik, değer aralığına göre sabit sayıda kutuya bölünür.
2. Bölünme araması, tüm benzersiz değerler yerine **kutu sınırları** üzerinden yapılır.
3. Bu, hem **bellek kullanımını** (sürekli değerler yerine kutu indeksleri saklanır) hem de **hesaplama süresini** (çok daha az sayıda aday bölünme noktası) önemli ölçüde azaltır.

**Ek Optimizasyon — Histogram Farkı (Histogram Subtraction):** Bir düğümün histogramı, kardeş düğümün ve ebeveynin histogramları biliniyorsa, çıkarma işlemiyle hızlıca hesaplanabilir ( $\text{kardeş} = \text{ebeveyn} - \text{diğer\_kardeş}$ ) — bu, her düğüm için histogramı sıfırdan hesaplama maliyetini yarıya indirir.

---

### 4. GOSS — Gradient-based One-Side Sampling

**Problem:** Boosting'in ilerleyen iterasyonlarında, çoğu örnek zaten iyi öğrenilmiştir (küçük gradyan) ve sadece az sayıda örnek hâlâ büyük hataya sahiptir (büyük gradyan). Tüm örnekleri her seferinde kullanmak, öğrenilmiş örnekler için gereksiz hesaplama israfına yol açar.

**GOSS'un Çözümü:**

1. Örnekler **gradyan büyüklüğüne göre** sıralanır.
2. **Büyük gradyanlı örneklerin tamamı** korunur (bunlar hâlâ “zor” örnekler, öğrenme için kritik).
3. **Küçük gradyanlı örneklerden ise rastgele bir alt küme** örneklenir (bunlar zaten “öğrenilmiş”, tam kullanmaya gerek yok).
4. Örneklemeye nedeniyle oluşan yanlılığı telafi etmek için, küçük gradyanlı örneklerin katkısına bir **düzeltilme katsayısı** uygulanır.

**Sonuç:** Veri setinin önemli bir kısmı örneklenerek dışarıda bırakılsa da, model doğruluğu neredeyse korunur — çünkü bilgi taşıyan (büyük gradyanlı) örnekler hiç kaybedilmez. Bu, eğitim hızını önemli ölçüde artırır.

## 5. EFB — Exclusive Feature Bundling

**Problem:** Gerçek dünya veri setlerinde (özellikle one-hot encode edilmiş kategorik değişkenlerden sonra) çok sayıda **seyrek (sparse)** özellik bulunur — bu özelliklerin çoğu zaman aynı anda sıfırdan farklı değer almadığı (**mutually exclusive**) görülür.

### EFB'nin Çözümü:

1. Birbiriyle nadiren aynı anda sıfırdan farklı olan özellikler tespit edilir (bir çakışma toleransı, tam olarak hiç çakışmama şartı aranmaz).
2. Bu özellikler **tek bir “paket” (bundle) özellikte** birleştirilir — her orijinal özelliğe farklı bir değer aralığı (offset) atanarak bilgi kaybı önlenir.
3. Bu sayede özellik sayısı azalır, histogram oluşturma ve bölünme arama süresi kısalmır.

**Sonuç:** Yüksek boyutlu, seyrek veri setlerinde (örn. çok sayıda kategorik değişken içeren veri) belirgin bellek ve hız kazancı sağlanır.

## 6. LightGBM'de Zaman Serisi Verisi — Kritik Bir Uyarı

LightGBM bir **tablo (tabular) modelidir**; zamanı kendiliğinden anlamaz. Zaman serisi problemlerinde (örn. hisse senedi, satış tahmini) şu kurallara **kesinlikle** uyulmalıdır:

- **Asla `train_test_split(..., shuffle=True)` kullanılmaz** — bu, gelecekteki bilginin geçmişe sızmasına (**data leakage**) neden olur ve modelin gerçekte olduğundan çok daha iyi görünmesine yol açar.
- Bölme **kronolojik** yapılmalıdır: geçmiş veri eğitim, en güncel dönem test seti olmalıdır.
- Hiperparametre optimizasyonu için `TimeSeriesSplit` kullanılmalı, standart k-fold **kullanılmamalıdır**.
- Daha sağlam değerlendirme için **walk-forward (kayan pencere) doğrulama** tercih edilir: model belirli bir noktaya kadar eğitilir, bir sonraki dönem test edilir, pencere kaydırılıp süreç tekrarlanır.
- Hedef değişken olarak doğrudan gelecekteki **fiyat** yerine **getiri (return)** kullanmak, modelin durağan (stationary) olmayan seriler yerine daha kararlı bir hedefe odaklanmasını sağlar.

Bu kurallar zaman serisi dışındaki standart sınıflandırma/regresyon problemlerinde geçerli değildir; orada standart stratified k-fold kullanılabilir.

## 7. Önemli Hiperparametreler — Sınav İçin Özet Tablo

| Parametre                  | Anlamı                                             | Etkisi                                                           |
|----------------------------|----------------------------------------------------|------------------------------------------------------------------|
| <code>num_leaves</code>    | Bir ağaçtaki maksimum yaprak sayısı                | Leaf-wise büyümede en kritik parametre; çok yüksekse overfitting |
| <code>max_depth</code>     | Maksimum derinlik (leaf-wise'ta ikincil bir sınır) | <code>num_leaves</code> ile birlikte overfitting kontrolü sağlar |
| <code>learning_rate</code> | Küçültme katsayısı                                 | Düşükse daha fazla ağaç gerekir, genelde daha iyi genelleme      |

| Parametre         | Anlamı                                 | Etkisi                                                    |
|-------------------|----------------------------------------|-----------------------------------------------------------|
| n_estimators      | Ağaç sayısı                            | Erken durdurma ile birlikte kullanılabilir                |
| min_child_samples | Bir yaprakta gereken min. örnek sayısı | Artınca overfitting azalır (küçük veri setlerinde önemli) |
| feature_fraction  | Sütun örnekleme oranı                  | Ağaçlar arası korelasyonu azaltır                         |
| bagging_fraction  | Satır örnekleme oranı                  | Varyansı azaltır, hızlandırır                             |
| max_bin           | Histogram kutu sayısı                  | Artınca hassasiyet artar ama hız/bellek maliyeti de artar |

## 8. LightGBM'in XGBoost ile Karşılaştırması

| Kriter                       | XGBoost                                | LightGBM                                                                  |
|------------------------------|----------------------------------------|---------------------------------------------------------------------------|
| Ağaç büyütme                 | Level-wise (katman katman)             | Leaf-wise (best-first)                                                    |
| Büyük veri hızı              | İyi                                    | Çok iyi (genelde daha hızlı)                                              |
| Küçük veri overfitting riski | Orta                                   | Leaf-wise nedeniyle biraz daha yüksek (dikkatli num_leaves ayarı gerekir) |
| Örnekleme tekniği            | Standart subsample                     | GOSS (gradyan tabanlı tek taraflı örnekleme)                              |
| Seyrek özellik yönetimi      | Sparsity-aware split (eksik veri için) | EFB (seyrek özellikleri paketleme)                                        |
| Bellek kullanımı             | Orta                                   | Düşük (histogram + bin tabanlı temsil)                                    |
| En kritik hiperparametre     | max_depth, gamma                       | num_leaves, min_child_samples                                             |

## 9. LightGBM'in Genel Ensemble/Boosting Ailesi İçindeki Yeri

| Kriter                 | Random Forest          | AdaBoost           | Gradient Boosting    | XGBoost                | LightGBM   |
|------------------------|------------------------|--------------------|----------------------|------------------------|------------|
| Eğitim stratejisi      | Paralel                | Sıralı             | Sıralı               | Sıralı                 | Sıralı     |
| Ağaç büyütme           | Bağımsız, tam derinlik | Kütük (1 seviye)   | Level-wise, sığ      | Level-wise + budama    | Leaf-wise  |
| Regularizasyon         | Sınırlı                | Yok                | Sınırlı              | L1/L2 (güçlü)          | L1/L2      |
| Büyük veri performansı | İyi                    | Orta               | Orta                 | İyi                    | Çok iyi    |
| Örnekleme yeniliği     | Bootstrap              | Ağırlık güncelleme | Stochastic subsample | Subsample + col-sample | GOSS + EFB |

## 10. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma

- LightGBM'in en ayırt edici özelliği **leaf-wise (best-first) ağaç büyütme** stratejisidir — level-wise'a göre daha doğru ama daha overfitting'e yatkın ağaçlar üretir.
- `num_leaves`, LightGBM'de `max_depth`'ten daha kritik bir hiperparametredir.
- **GOSS**: büyük gradyanlı örnekler korunur, küçük gradyanlılardan örneklenir → eğitim hızlanır, doğruluk korunur.
- **EFB**: birbiriyle nadiren aynı anda sıfırdan farklı olan seyrek özellikler tek bir özellikte birleştirilir → boyut ve bellek azalır.
- Histogram tabanlı bölünme arama, sürekli değişkenleri kutulara (bin) ayırarak hem hızı hem bellek verimliliğini artırır.
- Zaman serisi verisinde **asla rastgele (shuffled) bölme kullanılmaz** — kronolojik bölme ve `TimeSeriesSplit/walk-forward` doğrulama şarttır (data leakage riski).
- LightGBM, XGBoost ile aynı temel matematiksel çerçeveyi paylaşır (gradyan + Hessian, regularizasyon); temel fark **ağaç büyütme stratejisi ve örnekleme teknikleridir (GOSS, EFB)**.
- Küçük veri setlerinde LightGBM'in leaf-wise büyümesi dikkatli ayarlanmazsa (küçük `num_leaves`, yüksek `min_child_samples`) overfitting riski taşır.